Engineering Document E03 — Backend Architecture Blueprint

Product Name: HQ

Engineering Package: Version 1.0

Purpose

This document defines the backend architecture for HQ.

The backend is responsible for authentication, business logic, AI orchestration, mission management, billing, analytics, notifications, storage, and secure communication with external services.

HQ adopts a modular architecture based on **NestJS**, **Prisma ORM**, and **Domain-Driven Design (DDD)** principles.

Engineering Goals

The backend should be:

- Modular
- Maintainable
- Testable
- Scalable
- Secure
- Event-driven
- Cloud-native
- AI-first

Business logic must remain independent from frameworks wherever practical.

Technology Stack

Framework

- NestJS

Language

- TypeScript

Database

- PostgreSQL

ORM

- Prisma ORM

Authentication

- Firebase Authentication

Validation

- class-validator
- class-transformer
- Zod (shared schemas where beneficial)

Queue

- BullMQ

Cache

- Redis

Storage

- Google Cloud Storage

Documentation

- Swagger/OpenAPI

Logging

- Pino

Testing

- Jest

Backend Architecture

```
API Request

↓

Guards

↓

Interceptors

↓

Validation Pipe

↓

Controller

↓

Application Service

↓

Domain Service

↓

Repository (Prisma)

↓

PostgreSQL

↓

Response
```

Each layer has a single responsibility.

Layer Structure

Presentation Layer

Responsible for:

- Controllers
- DTOs
- Request validation
- Authentication
- Authorization
- HTTP responses

No business logic belongs here.

Application Layer

Responsible for:

- Use cases
- Service orchestration
- Transaction coordination
- Event publishing

This layer coordinates business operations.

Domain Layer

Contains:

- Entities
- Value Objects
- Domain Services
- Business Rules
- Domain Events

This is the heart of HQ.

Infrastructure Layer

Responsible for:

- Prisma
- Firebase
- Gemini API
- Redis
- Storage
- Email
- External APIs

Framework-specific code belongs here.

Module Structure

```
apps/api/src/

modules/

shared/

common/

config/

main.ts
```

Every business capability becomes an independent module.

Core Modules

Authentication Module

Organization Module

User Module

Executive Module

Mission Module

AI Module

Prompt Module

Memory Module

Billing Module

Analytics Module

Storage Module

Notification Module

Audit Module

Admin Module

Health Module

Configuration Module

Shared Module

Contains reusable functionality.

Examples:

- Exceptions
- Base classes
- Interfaces
- Utilities
- Constants
- Decorators
- Helpers

Common Module

Contains framework-level functionality.

Examples:

- Guards
- Pipes
- Filters
- Interceptors
- Middleware

Dependency Rules

Controllers

↓

Application Services

↓

Domain Services

↓

Repositories

↓

Database

Dependencies must always point inward.

Circular dependencies are prohibited.

Repository Pattern

Repositories abstract Prisma.

Example:

MissionRepository

ExecutiveRepository

OrganizationRepository

PromptRepository

Controllers never access Prisma directly.

Dependency Injection

NestJS Dependency Injection manages:

- Services
- Repositories
- AI providers
- Storage providers
- Queue services
- Configuration

Concrete implementations remain replaceable.

CQRS Strategy

Use CQRS where it improves clarity.

Commands:

- Create Mission
- Execute Mission

- Upgrade Subscription

Queries:

- Get Dashboard
- List Missions
- Get Analytics

Simple CRUD operations need not use CQRS.

Event-Driven Design

Important events include:

MissionCreated

MissionCompleted

SubscriptionActivated

ExecutiveAssigned

AssetGenerated

UserRegistered

Events enable loose coupling between modules.

Queue Architecture

BullMQ processes background work.

Examples:

- AI generation
- Email delivery
- Asset processing
- Analytics aggregation
- Notifications

Heavy operations should never block HTTP requests.

AI Integration

AI requests pass through a dedicated AI Gateway.

Responsibilities:

- Prompt assembly
- Context injection
- Provider communication
- Response validation
- Retry logic

No module should call an AI provider directly.

Configuration

Environment variables are centralized.

Categories:

- Database
- Firebase
- Gemini
- Redis
- Storage
- Billing
- Email

Configuration validation occurs during startup.

Security

Security includes:

- JWT verification
- Firebase token validation
- RBAC
- Rate limiting
- Input validation
- CORS
- Helmet
- Secure headers

Security is enforced globally.

Error Handling

Global Exception Filter manages:

- Validation errors
- Business errors
- Infrastructure errors
- External API failures

Errors should be consistent and machine-readable.

Logging

Every request logs:

- User ID
- Organization ID
- Endpoint
- Duration
- Status
- Correlation ID

Sensitive information must never be logged.

Validation

Incoming requests pass through:

- DTO validation
- Schema validation
- Business rule validation

Invalid data should never reach the domain layer.

API Versioning

Support:

v1

Future versions should coexist without breaking existing clients.

File Upload Pipeline

Files flow through:

Validation

↓

Virus Scan (future)

↓

Storage

↓

Metadata Save

↓

Asset Registration

↓

Response

Caching Strategy

Redis caches:

- Organization settings
- Feature flags
- Frequently accessed prompts
- Dashboard summaries

Cache invalidation follows write operations.

Health Monitoring

Expose health endpoints for:

- Database
- Redis
- AI Provider
- Storage
- Queue
- Overall application status

These support operational monitoring.

Testing Strategy

Every module requires:

- Unit tests

- Integration tests
- API tests

Critical workflows should include end-to-end tests.

Coding Standards

Modules must include:

- Controller
- Service
- Repository
- DTOs
- Tests
- Documentation

Folder structures should remain consistent across modules.

Scalability

The architecture supports:

- Horizontal scaling
- Multiple AI providers
- Distributed queues
- Additional microservices
- Enterprise features

Growth should not require redesigning the core architecture.

Success Criteria

The Backend Architecture succeeds when:

- Modules remain independent.

- Business logic is isolated.
- AI integrations are centralized.
- Performance remains predictable.
- New features can be added with minimal coupling.
- The backend supports both the hackathon MVP and future enterprise growth.

Conclusion

The HQ Backend Architecture provides a production-ready foundation built on NestJS, Prisma ORM, PostgreSQL, Redis, and Google Cloud services.

By combining modular design, Domain-Driven Design principles, event-driven communication, and a dedicated AI orchestration layer, HQ achieves a backend architecture that is maintainable, scalable, and capable of supporting sophisticated AI-powered business workflows.